

RAPPORT DE PROJET DE FIN D'ETUDES

Présenté en vue de l'obtention du

Diplôme National de Master en Sciences de l'Informatique

Spécialité : Génie Logiciel et Systèmes d'Information

YOUR PROJECT TITLE HERE

Subtitle if applicable

Par

Louai BOUBAKER

Encadrant professionnel : Madame Bakhta Haouari

Encadrant académique : Madame Dhikra Ben Mahmoud

Réalisé au sein de



Chapter I

PROJECT GENERAL FRAMEWORK

1 Introduction

This chapter provides an overview of the context surrounding our graduation project, beginning with a presentation of the host organization, E-TAFAKNA, where we developed our solution. We will analyze the specific issues that motivated this initiative, then explain the original idea we proposed. Finally, we will summarize the detailed procedures we followed to ensure the smooth progress and successful completion of the project.

2 Host Organization Presentation

Founded in 2022, E-Tafakna stands as a trailblazing Tunisian LegalTech startup transforming legal document management across the Middle East and North Africa (MENA) region [1]. Through its intuitive web and mobile platform, the company enables users to seamlessly create, collaborate on, sign, and monitor legal documents from any location, effectively digitizing the entire contract lifecycle.



Figure 1.1: E-Tafakna Company Logo [2]

E-Tafakna distinguishes itself through a deeply user-centric philosophy. The platform offers over 100 pre-designed contract templates covering everything from non-disclosure agreements to employment contracts, coupled with real-time document status tracking and trilingual support in Arabic, French, and English. This strategic combination of simplicity and

robust functionality democratizes access to professional legal services, making them affordable and accessible for freelancers, entrepreneurs, and Small and Medium-sized Enterprise (SME)s. Recognized with the prestigious StartUp’Act label and winner of the HiiL Innovating Justice Challenge 2024, E-Tafakna continues to solidify its position as a regional leader in legal innovation [1].

3 General Project Overview

3.1 Project Framework

Legal services have long been associated with complexity, high costs, and limited accessibility, particularly for individuals, freelancers, and small businesses navigating an increasingly regulated environment. The digitalization of these services has become a strategic priority, as modern users expect instant, accurate, and accessible legal support. In this context, Artificial Intelligence (AI) is emerging as a transformative force, enabling platforms to automate complex legal tasks, centralize expertise, and deliver seamless experiences across multiple channels.

E-Tafakna addresses this need through a comprehensive legal platform that covers the entire contract lifecycle. Its services span contract template creation, professional consulting, document management, real-time contract collaboration, and electronic signature. To further enhance this offering, E-Tafakna has integrated two intelligent AI agents: Elyssa, a smart legal co-pilot focused on document drafting and compliance, and Kahina, an intelligent legal avatar designed to explain laws, contracts, and regulations through natural conversation. Together, these agents position E-Tafakna as a pioneer in AI-powered LegalTech across the MENA region.

3.2 Study of Existing Solutions

E-Tafakna currently offers two AI-powered agents to assist users with their legal needs.

Elyssa is a smart legal co-pilot designed to assist users throughout the document drafting and review process. It provides:

- **Legal Chatbot:** Instant legal assistance through a conversational interface.
- **Clause Assistant:** Smart generation of contract clauses tailored to the user’s context.
- **Contract Analyzer:** Automatic identification of risks and missing clauses within uploaded documents.
- **Compliance Scorer:** Rates contracts against legal standards and suggests targeted improvements.

Kahina is an intelligent legal avatar built to make legal knowledge accessible to SMEs, freelancers, and startups. It provides:

- **Legal Intelligence:** Helps users understand legal concepts, procedures, and applicable regulations.
- **Virtual Assistant:** Delivers contextual guidance tailored to the user’s specific questions.
- **Multilingual Support:** Communicates fluently in Arabic, French, and English.
- **Conversational Avatar:** Simulates human-like legal consultations for a more natural user experience.

Together, Elyssa and Kahina form the intelligent core of E-Tafakna’s platform, each serving a distinct but complementary role: Elyssa focuses on document-level assistance, while Kahina handles broader legal understanding and user guidance [2].

3.3 Critical Analysis of the Existing System

While Elyssa and Kahina represent meaningful advances in accessible legal assistance, a critical analysis reveals that the current system still falls short of its full potential. Kahina’s conversational avatar interface has already addressed the interaction barriers previously faced by users, offering multilingual dialogue in Arabic, French, and English. However, two fundamental limitations remain unresolved:

- **Lack of Tunisian Legal Specificity:** Both Elyssa and Kahina rely on general-purpose language models that were not trained on Tunisian legal texts. As a result, responses often lack the precision required for local legal matters, missing the nuances of Tunisian legislation, procedures, and regulatory frameworks.
- **Dependency on External Cloud Services:** The current AI infrastructure relies on external cloud-based Application Programming Interface (API)s, which raises concerns around data sovereignty, response latency, and compliance with Tunisian data regulations. E-Tafakna’s planned deployment at Agence Tunisienne d’Internet (ATI) Tunisie requires a fully on-premises solution with no external dependencies, ensuring that all legal data remains within a secure and controlled local infrastructure — a requirement the current system cannot fulfill.

These two limitations — insufficient Tunisian legal knowledge and reliance on external cloud services — define the core technical challenge that our project aims to address..

3.4 Problem Statement

The analysis of E-Tafakna’s existing AI agents reveals a clear and twofold challenge. First, both Elyssa and Kahina operate on general-purpose language models that lack the domain-specific knowledge required to handle Tunisian legal content with the accuracy and reliability that legal matters demand. Second, the current architecture depends on external cloud services, making it incompatible with the on-premises deployment requirements of ATI Tunisie, where data sovereignty and infrastructure autonomy are non-negotiable constraints.

This raises a central research question: *How can we build a legal AI assistant that is both deeply grounded in Tunisian law and fully deployable within a secure, self-contained local infrastructure?*

Addressing this question requires tackling three concrete technical problems:

- **Domain Adaptation:** General-purpose language models must be specialized through fine-tuning on a curated corpus of Tunisian legal texts, enabling them to understand the specific terminology, structure, and logic of local legislation.
- **Knowledge Retrieval:** Fine-tuning alone is insufficient for handling the breadth and evolving nature of legal knowledge. A Retrieval-Augmented Generation (RAG) pipeline must be designed to allow the model to dynamically retrieve relevant legal documents at inference time, improving both accuracy and up-to-date coverage.
- **Local Deployment:** The resulting system must be optimized to run efficiently on on-premises infrastructure at ATI Tunisie, without relying on any external API or cloud service, ensuring full data sovereignty and regulatory compliance.

3.5 Proposed Solution

To address the identified challenges, we propose the development of a domain-specific legal AI assistant built around two complementary components: a fine-tuned Small Language Model (SLM) or Large Language Model (LLM) specialized in Tunisian law, and a RAG pipeline designed to dynamically incorporate legal knowledge at inference time. The resulting system will be deployed entirely on-premises at ATI Tunisie, with no dependency on external cloud services.

Component 1: Fine-Tuned Language Model

Rather than relying on a generic pre-trained model, we will fine-tune a base SLM or LLM on a curated corpus of Tunisian legal documents, including legislation, regulatory texts, and legal procedures. This supervised fine-tuning process will enable the model to internalize the specific vocabulary, structure, and reasoning patterns of Tunisian law, producing responses that are more accurate, contextually appropriate, and legally sound than those of a general-purpose model.

Component 2: Retrieval-Augmented Generation Pipeline

Fine-tuning alone cannot capture the full breadth of Tunisian legal knowledge, nor can it account for legislative updates over time. To overcome this, we will design a RAG pipeline that connects the language model to a vector database populated with indexed legal documents. At inference time, the system will retrieve the most relevant legal excerpts based on the user’s query and inject them into the model’s context, significantly improving the precision and coverage of generated responses.

Component 3: On-Premises Deployment at ATI Tunisie

The complete system — language model, RAG pipeline, and vector database — will be deployed locally on ATI Tunisie’s infrastructure using Ollama as the local LLM inference

framework. This architecture guarantees full data sovereignty, eliminates latency caused by external API calls, and ensures compliance with Tunisian data regulations. All legal data processed by the system remains within ATI’s secure on-premises environment.

Expected Outcomes

This solution is expected to deliver the following improvements over the existing system:

- **Higher Legal Accuracy:** Domain-specific fine-tuning ensures the model understands and correctly applies Tunisian legal concepts, reducing erroneous or imprecise responses.
- **Up-to-Date Legal Knowledge:** The RAG pipeline allows the system to retrieve current legal documents dynamically, keeping responses relevant even as legislation evolves.
- **Full Data Sovereignty:** On-premises deployment at ATI Tunisie ensures that no legal data leaves the organization’s infrastructure, meeting both internal security requirements and national data regulations.
- **Seamless Integration:** The system is designed to integrate with E-Tafakna’s existing platform, enhancing the capabilities of both Elyssa and Kahina without requiring changes to the user-facing interface.

Together, these components form a cohesive and technically grounded solution that directly addresses each of the three problems identified in the problem statement, transforming E-Tafakna’s legal AI offering into a specialized, autonomous, and locally sovereign system.

4 Adopted Methodology

Choosing the right methodology is one of the most consequential decisions in any technical project. It determines not only how the work is organized, but how decisions are made, how progress is measured, and how results are ultimately validated. In the context of an AI-driven system designed for a real-world legal environment, this choice carries particular weight.

4.1 Methodological Framework Selection

A wide range of methodological frameworks exists for guiding the development of software and intelligent systems. These can be broadly grouped into two families: traditional project management methodologies, and data science or research-oriented methodologies.

Traditional project management methodologies, such as Scrum and Kanban, are general-purpose frameworks focused on how a team organizes and delivers work. Scrum structures development into fixed-length iterations called sprints, with regular reviews and adaptation cycles, while Kanban emphasizes continuous flow and visual task management. Both are well-suited for software delivery projects where requirements are relatively stable and the primary challenge is coordination and delivery pace. However, they do not address the specific challenges inherent to AI development — they provide no guidance on data collection, model training, evaluation of predictive performance, or the construction and

validation of a research artifact. Applying them alone to a project involving fine-tuning a language model and building a RAG pipeline would leave the most critical technical and scientific dimensions of the work entirely unguided.

Data science and research-oriented methodologies, by contrast, are designed precisely for projects where the core challenge is building, evaluating, and deploying an intelligent system or a research artifact. In the field of data science and intelligent systems design, several such frameworks have been proposed. These range from traditional process-oriented approaches, such as Cross-Industry Standard Process for Data Mining (CRISP-DM) [3] and Team Data Science Process (TDSP) [4], to research-centered paradigms such as Design Science Research (DSR) [5], [6]. Each addresses a different type of project and carries distinct strengths and limitations that must be carefully weighed against the specific requirements at hand.

Our project presents a specific set of requirements that constrain this choice:

- **Domain Specialization:** Fine-tuning a language model on Tunisian legal texts requires multiple cycles of data preparation, training, and evaluation before acceptable performance is achieved.
- **Pipeline Complexity:** Integrating a fine-tuned model with a RAG pipeline and a vector database introduces interdependencies between components that must be developed, tested, and refined in coordination.
- **Deployment Constraints:** The on-premises deployment at ATI Tunisie imposes strict infrastructure requirements that must be addressed from the earliest stages of the project.
- **Artifact-Centered Development:** The primary deliverable of this project is a concrete technological artifact — a specialized legal chatbot — whose utility must be rigorously designed, built, and evaluated within an academic framework.

The following sections present an overview of each candidate methodology and the rationale behind our final selection.

4.2 Overview of CRISP-DM

CRISP-DM [3] is a traditional process model that emerged in the late 1990s to standardize the practice of data mining — the automated extraction of patterns and knowledge from large datasets. It was conceived as an industry-neutral framework applicable across sectors and tools, structured around six sequential phases [3]:

- **Business Understanding:** Defining project objectives and requirements from a business perspective.
- **Data Understanding:** Collecting initial data and identifying quality issues and relevant subsets.
- **Data Preparation:** Cleaning, transforming, and structuring data for modeling.

- **Modeling:** Applying modeling techniques and calibrating parameters for optimal results.
- **Evaluation:** Assessing the model against business objectives.
- **Deployment:** Integrating the model into operational systems.

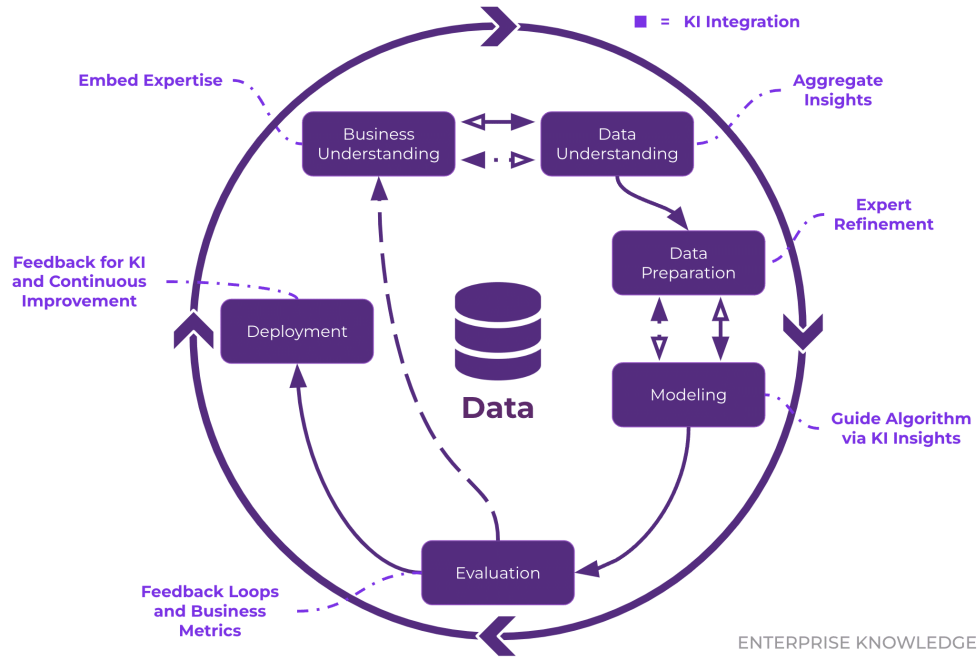


Figure 1.2: CRISP-DM Methodology Phases [3]

While CRISP-DM has been widely adopted in traditional data mining and analytics projects, its linear progression and narrow focus on pattern extraction from existing datasets make it ill-suited for projects that involve designing and deploying an intelligent system from the ground up. It provides no framework for artifact construction, formal evaluation of system utility, or academic communication of results [3].

4.3 Overview of TDSP

TDSP [4] is a more recent methodology developed specifically for AI and machine learning projects in enterprise environments. First released in 2016, it combines agile principles with a structured data science lifecycle to address the challenges of building intelligent applications intended for production deployment.

The methodology is built on four core components [4]:

- **Data Science Lifecycle:** An iterative process guiding projects from business understanding through to deployment and continuous improvement.

- **Standardized Project Structure:** Consistent templates and folder organization that promote team collaboration and reproducibility.
- **Infrastructure Recommendations:** Guidelines for cloud resources, development environments, and deployment platforms.
- **Tool and Utility Guidance:** A curated selection of frameworks and libraries supporting the entire AI development pipeline.

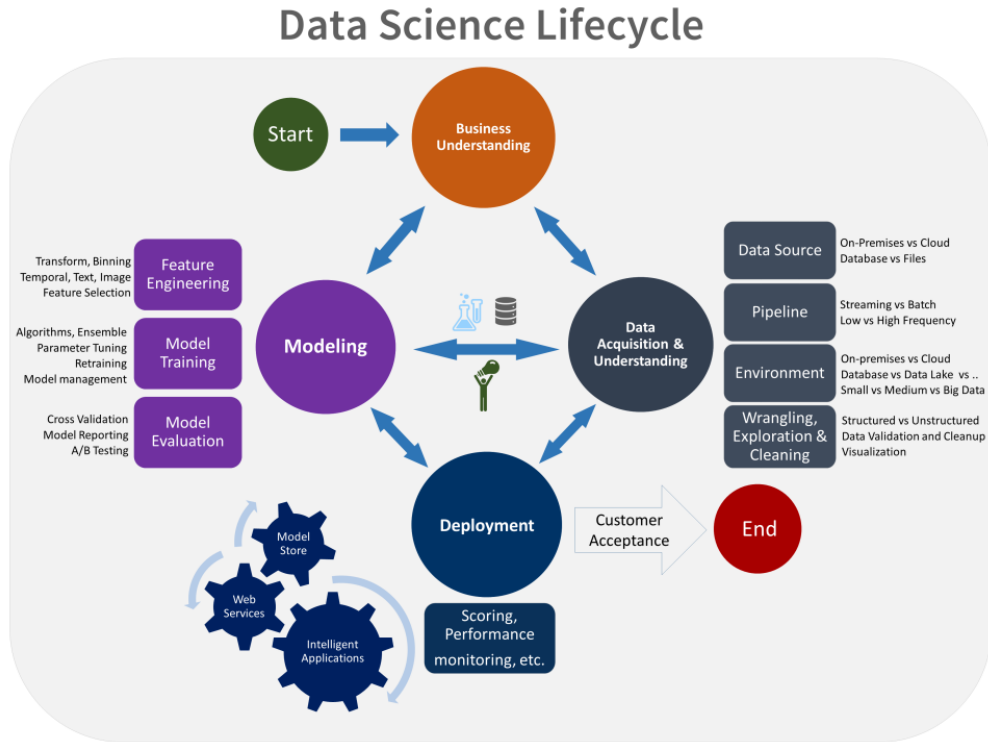


Figure 1.3: TDSP Iterative Lifecycle [4]

Although TDSP represents a significant improvement over CRISP-DM for modern AI projects, it remains fundamentally an industry-oriented engineering framework. It does not provide a formal mechanism for artifact-centered research, rigorous academic evaluation, or scientific communication of results — all of which are essential requirements for an academic graduation project [4].

4.4 Overview of DSR

DSR [5], [6] is a well-established research paradigm in information systems and computer science, specifically designed for projects whose primary output is a technological artifact — a system, model, method, or tool — built to solve a clearly identified real-world problem. Unlike traditional process methodologies, DSR is rooted in a scientific approach: it requires

not only that an artifact be constructed, but that its utility and effectiveness be formally evaluated and communicated as a research contribution [5].

The DSR process model [6] consists of six iterative activities:

- **Problem Identification and Motivation:** Defining the research problem and justifying the value of developing a solution.
- **Definition of Objectives:** Specifying what a successful artifact must achieve, derived directly from the problem definition.
- **Design and Development:** Constructing the artifact, encompassing data collection, model training, system architecture, and integration.
- **Demonstration:** Deploying and using the artifact to solve the identified problem in a realistic setting.
- **Evaluation:** Assessing how well the artifact addresses the problem using appropriate metrics, comparisons, and validation techniques.
- **Communication:** Disseminating the problem, the artifact, and the findings to relevant audiences through academic reporting and presentation.

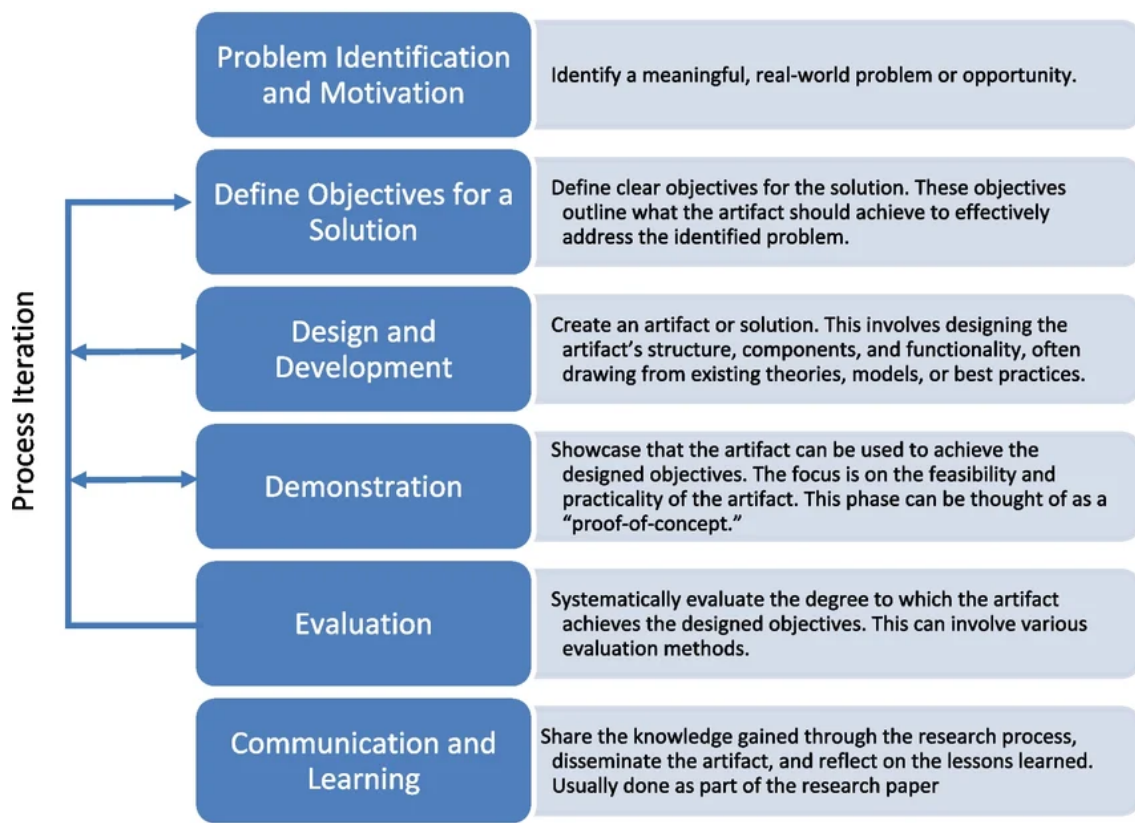


Figure 1.4: DSR Process Model [6]

4.5 Methodologies Comparison

Table I.1 presents a structured comparison of the three methodologies across criteria relevant to the specific requirements of our project.

Table I.1: Comparative Analysis of CRISP-DM, TDSP, and DSR

Criteria	CRISP-DM	TDSP	DSR
Primary Focus	Data mining and pattern extraction	Production AI and ML systems	Design and evaluation of technological artifacts
Development Approach	Linear, sequential	Agile, iterative	Iterative, artifact-centered
Academic Rigor	Low	Low	High
Evaluation Framework	Basic model metrics	Deployment monitoring	Formal utility and efficacy assessment
Artifact Orientation	None	Partial	Central
Communication Phase	Not included	Not included	Explicitly included
Deployment Guidance	Basic system integration	Detailed production recommendations	Demonstration in realistic environment
Suitability for PFE	Weak	Moderate	Strong

4.6 Why We Chose DSR

The comparative analysis reveals that DSR [5], [6] is the methodology best aligned with the nature and objectives of our project, for the following reasons:

- **Artifact-Centered:** Our project’s primary deliverable is a specialized legal chatbot — a concrete technological artifact. DSR is built precisely around the design, construction, and evaluation of such artifacts [5].
- **Academic Rigor:** Unlike CRISP-DM and TDSP, DSR provides a formal evaluation framework that allows the utility and performance of the system to be rigorously assessed — an essential requirement for an academic graduation project.
- **Structured Evaluation:** The dedicated evaluation phase enables a principled comparison between the fine-tuned model and a generic baseline, providing scientifically grounded evidence of improvement [6].
- **Communication Phase:** DSR explicitly includes a communication activity that maps directly onto the writing of this report and its oral defense, making the methodology fully consistent with the academic context of this work.

- **Iterative by Design:** The build-and-evaluate cycle at the core of DSR [5] naturally accommodates the multiple iterations of fine-tuning, testing, and refinement that domain adaptation of a language model requires.

4.7 Our Adapted DSR Lifecycle

We tailored the DSR lifecycle [6] to fit the specific structure of our project, embedding the full AI technical pipeline within the Design and Development phase:

Phase 1 — Problem Identification and Motivation: We identified the core limitations of E-Tafakna’s existing AI agents — insufficient Tunisian legal knowledge and dependency on external cloud services — and established the need for a domain-specific, locally deployable solution.

Phase 2 — Definition of Objectives: We defined three concrete objectives for the artifact: domain adaptation through fine-tuning on Tunisian legal corpora, dynamic knowledge retrieval through a RAG pipeline, and full on-premises deployment at ATI Tunisie.

Phase 3 — Design and Development: This phase encompasses the complete AI technical pipeline:

- Collection and preprocessing of Tunisian legal documents.
- Fine-tuning of the base SLM or LLM on legal data.
- Design and implementation of the RAG pipeline and vector database.
- Integration of the complete system into E-Tafakna’s platform architecture.

Phase 4 — Demonstration: The artifact is deployed on ATI Tunisie’s on-premises infrastructure and demonstrated through realistic legal consultation scenarios covering the use cases of both Elyssa and Kahina.

Phase 5 — Evaluation: The system is evaluated using quantitative metrics — including response accuracy, legal precision, and retrieval quality — and compared against a generic baseline model to measure the concrete contribution of domain-specific fine-tuning.

Phase 6 — Communication: The findings, artifact design, and evaluation results are communicated through this academic report and its oral defense before the graduation jury.

5 Choice of Services, Technologies, and Tools

Beyond selecting a methodological framework, the successful execution of our project required identifying the right set of services and technologies to support its different phases. This section presents the main platform we relied on during the design and development of our solution, and explains the rationale behind its selection.

5.1 Azure AI Foundry: Access to State-of-the-Art Language Models

Azure AI Foundry is Microsoft’s unified platform for building, evaluating, and deploying generative AI applications. It provides centralized access to a catalog of foundation models from multiple providers — including OpenAI, Meta, and Mistral — through a single secure endpoint, along with tooling for prompt engineering, model evaluation, and enterprise-grade governance [7].



Figure 1.5: Azure AI Foundry Platform [7]

Within this ecosystem, we specifically relied on the **GPT-4.1** model made available through Azure OpenAI Service. GPT-4.1 is one of the most advanced large language models offered on the platform, featuring an expanded context window, strong multimodal capabilities, and improved instruction following compared to previous generations — characteristics that are particularly valuable when processing dense and heterogeneous legal documents [8].

Role in Our Project

GPT-4.1 played a central role in the data preparation phase of our project. The Tunisian legal corpus required for fine-tuning was originally available as a large collection of PDF documents — including legislation, official gazettes, and regulatory texts — whose content was often embedded in complex layouts, multi-column formats, and scanned pages. Traditional text extraction tools proved insufficient to reliably capture the structure and semantics of these documents.

To overcome this limitation, we used GPT-4.1 as an intelligent extraction engine capable of:

- **Accurate Text Extraction:** Converting raw PDF content into clean, structured text while preserving the hierarchical organization of legal documents (articles, sections, and

clauses).

- **Semantic Structuring:** Identifying and labeling key components of legal texts, such as article numbers, titles, and references to other legal provisions.
- **Noise Removal:** Filtering out irrelevant artifacts introduced by the PDF format, including headers, footers, page numbers, and watermarks.

The output of this extraction process served as the foundation of our curated Tunisian legal dataset, which was subsequently used for fine-tuning the target language model and populating the vector database of the RAG pipeline.

Rationale for Selection

Azure AI Foundry was chosen for its enterprise-grade security guarantees, its native integration with other Azure services, and its compliance with data protection standards — all of which align with the confidentiality requirements of legal data handling. Although the final production system is deployed entirely on-premises at ATI Tunisie, leveraging GPT-4.1 during the data preparation phase allowed us to build a high-quality legal corpus efficiently, without compromising the on-premises nature of the final deliverable.

5.2 Visual Studio Code

Visual Studio Code is a free, lightweight code editor maintained by Microsoft. While it remains light on system resources, it can be extended to support virtually any language or workflow through its extension marketplace, and brings together editing, debugging, version control, and a built-in terminal within a single interface [9].



Figure 1.6: Visual Studio Code [9]

Role in Our Project

VS Code acted as the single workspace where most of the technical work took place. Its versatility allowed us to handle several aspects of development without leaving the editor:

- **Python Development:** Writing and debugging the data preparation routines, the fine-tuning scripts, and the RAG integration code, with the help of the official Python extension and interactive Jupyter notebooks.
- **Version Control:** Managing the project's source code directly through the built-in Git interface, which made tracking changes and navigating the project history straightforward.

- **Integrated Terminal:** Launching training jobs, installing dependencies, and running command-line utilities from within the editor itself, avoiding the need to jump between different windows.

Rationale for Selection

We chose VS Code mainly for practical reasons: it runs smoothly on modest hardware, supports the full Python ecosystem we needed, and brings editing, debugging, and version control together in one interface. Keeping everything in a single workspace noticeably reduced the friction of switching between tasks, which matters in a project that combines data handling, model training, and integration work.

5.3 Python

Python is a high-level, interpreted programming language known for its clear syntax and the maturity of its scientific and AI libraries. It is today the most widely adopted language in the AI and machine learning community, largely because of the richness of its ecosystem around data processing, model training, and evaluation [10].



Figure 1.7: Python Programming Language [10]

Role in Our Project

Python served as the main language for every technical component of our solution. Its rich ecosystem allowed us to cover the entire AI pipeline without having to mix multiple languages:

- **Data Processing:** Cleaning and restructuring the Tunisian legal corpus extracted from Portable Document Format documents, using libraries such as Pandas and NumPy.
- **Model Fine-Tuning:** Building the fine-tuning pipeline around widely used AI frameworks, including Hugging Face Transformers and PyTorch.
- **RAG Pipeline Development:** Implementing the different stages of the retrieval-augmented generation workflow, from embedding generation to vector store queries and orchestration of the overall flow.
- **Scripting and Automation:** Writing smaller utility scripts to automate recurring tasks such as dataset preparation, evaluation runs, and aggregation of results.

Rationale for Selection

Python was chosen for its central role in the AI ecosystem and the availability of mature libraries covering every stage of our pipeline — from data extraction to model training, evaluation, and deployment. Its readability also made the codebase easier to document and maintain, which was particularly valuable in the academic context of this project.

5.4 n8n

n8n is an open-source workflow automation platform that allows users to design processing pipelines visually, by connecting nodes that each perform a specific action — from calling an API to reading a file or running a script. It can be fully self-hosted, meaning workflow data never has to leave the local environment [11].



Figure 1.8: n8n Workflow Automation Platform [11]

Role in Our Project

n8n was used to orchestrate the different stages of our processing pipeline. Rather than chaining scripts manually or writing a dedicated runner, we built workflows that trigger each step in the right order and give us a clear visual overview of the whole process:

- **Data Pipeline Orchestration:** Coordinating the sequence of operations involved in preparing the legal corpus, from raw document ingestion to structured output generation.
- **API Integration:** Connecting the different components of the system through Hypertext Transfer Protocol calls, which simplified communication between services without adding custom glue code.
- **Process Automation:** Scheduling and triggering recurring tasks automatically, reducing manual work and keeping pipeline executions consistent.

Rationale for Selection

n8n was selected mainly because it is open-source and self-hostable, which aligns with the on-premises deployment constraints imposed by ATI Tunisie. Beyond that, its visual approach lowered the overhead of managing a growing number of processing steps, and the ability to keep everything on local infrastructure ensured that no data ever left the controlled environment during automated processing.

5.5 FastAPI

FastAPI is a modern Python web framework used to expose code as Hypertext Transfer Protocol services. It is designed around Python type hints, which it leverages to validate requests automatically and generate interactive documentation, while offering strong performance thanks to its native support for asynchronous operations [12].



Figure 1.9: FastAPI Framework [12]

Role in Our Project

FastAPI served as the bridge between our Python scripts and the n8n automation platform. Since n8n interacts with external services through Hypertext Transfer Protocol calls, we wrapped our extraction and RAG-related scripts behind small FastAPI endpoints so that n8n workflows could invoke them like any other integration:

- **Script Exposition:** Turning standalone Python scripts — including the legal data extraction routines and the RAG pipeline components — into Hypertext Transfer Protocol endpoints that could be consumed directly from n8n.
- **Seamless Orchestration:** Allowing n8n to trigger Python operations as part of larger automated workflows, keeping a clean separation between orchestration on one side and execution on the other.
- **Automatic Validation and Documentation:** Relying on FastAPI's type-based validation and auto-generated interactive interface to speed up the development and testing of each endpoint.

Rationale for Selection

FastAPI was chosen because it makes exposing Python functions as web endpoints almost trivial, while still offering solid performance under the kind of long-running inference calls our pipeline involves. Its native support for asynchronous operations fit naturally with our AI workloads, and its tight integration with the Python ecosystem made it the most practical choice to connect our processing scripts to the n8n automation layer.

6 Conclusion

This chapter established the general framework of our graduation project. We began by presenting E-Tafakna, a pioneering Tunisian LegalTech startup operating across the MENA region, whose platform digitizes the entire legal document lifecycle and integrates two AI-powered agents — Elyssa and Kahina — to assist users with document drafting, compliance, and legal guidance.

A critical analysis of the existing system revealed two fundamental limitations: the reliance on general-purpose language models that lack grounding in Tunisian legal knowledge, and a dependency on external cloud services that is incompatible with the on-premises deployment requirements of ATI Tunisie. These limitations gave rise to a central research question — how to build a legal AI assistant that is both deeply specialized in Tunisian law and fully deployable within a secure, self-contained local infrastructure.

In response, we proposed a solution built around three complementary components: a fine-tuned SLM or LLM trained on Tunisian legal corpora, a RAG pipeline for dynamic legal knowledge retrieval, and a fully on-premises deployment at ATI Tunisie using a local inference framework.

Finally, after evaluating several methodological frameworks — from traditional project management approaches such as Scrum and Kanban, to data science methodologies such as CRISP-DM and TDSP, and research-oriented paradigms — we adopted DSR as our guiding methodology. Its artifact-centered structure, formal evaluation framework, and explicit communication phase make it the most appropriate choice for a project that is both technically ambitious and academically rigorous.

The following chapter will build on this foundation by providing a detailed review of the state of the art, examining the key concepts, existing approaches, and technologies that underpin our proposed solution.

Bibliography

- [1] WeAreTech, *E-tafakna: Tunisian legaltech startup*, <https://wearetech.tn/>, Accessed: 2026-04-10, 2024.
- [2] E-Tafakna, *E-tafakna official website*, <https://etafakna.com/>, Accessed: 2026-04-10, 2024.
- [3] P. Chapman et al., “CRISP-DM 1.0: Step-by-step data mining guide,” SPSS Inc., Tech. Rep., 2000.
- [4] Microsoft, *Team data science process documentation*, <https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview>, Accessed: 2026-04-10, 2016.
- [5] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [6] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [7] Microsoft, *Azure ai foundry documentation*, <https://learn.microsoft.com/en-us/azure/ai-foundry/>, Accessed: 2026-04-10, 2024.
- [8] Microsoft, *Azure openai service models*, <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models>, Accessed: 2026-04-10, 2024.
- [9] Microsoft, *Visual studio code documentation*, <https://code.visualstudio.com/docs>, Accessed: 2026-04-10, 2024.
- [10] Python Software Foundation, *Python language reference*, <https://docs.python.org/3/>, Accessed: 2026-04-10, 2024.
- [11] n8n, *N8n documentation: Workflow automation tool*, <https://docs.n8n.io/>, Accessed: 2026-04-10, 2024.
- [12] S. Ramírez, *Fastapi documentation*, <https://fastapi.tiangolo.com/>, Accessed: 2026-04-10, 2024.